AMRITA SCHOOL OF COMPUTING

Machine Learning in CYS · 24CYS214

LABORATORY ASSIGNMENT

Experimenting with Word Vectorizer Models

Analyzing Trade-offs Between Performance and Time Complexity

Subject	Machine Learning in CYS (24CYS214)
Assignment Type	Laboratory Experiment
Dataset	IMDB Top 1000 Movies & TV Shows
Models Compared	Bag of Words (BoW) | TF-IDF
Classifier	Linear Support Vector Classifier (LinearSVC)

PART 1 — THEORETICAL BACKGROUND

1.1 Word Vectorization

Word vectorization is the process of converting textual data (words, phrases, or documents) into numerical vectors so that machine learning and deep learning models can process and analyze natural language effectively.

Need for Word Vectorization

Computers cannot understand human language directly — they operate on numbers. Word vectorization bridges this gap by transforming text into a mathematical format while preserving semantic meaning and relationships between words.

Primary Functions

- Numerical Representation of Text — converts raw text into machine-readable numerical form.
- Capturing Semantic Meaning — represents similar words with similar vectors.
- Preserving Word Relationships — encodes context and analogies between terms.
- Handling Contextual Information — advanced methods allow words to have different meanings in different sentences.
- Enabling Machine Learning Models — feeds structured input to classifiers and deep networks.
- Supporting Various NLP Applications — classification, translation, chatbots, and more.

1.2 Frequency-Based Methods

Frequency-based methods are traditional word vectorization techniques that represent text using the frequency of words in a document or corpus. These methods convert text into numerical form based on how often words appear. The two main types are Bag of Words (BoW) and TF-IDF.

1.2.1 Bag of Words (BoW)

Bag of Words is a simple word vectorization technique that represents text by counting the frequency of each word in a document, ignoring grammar and word order. It creates a vocabulary of unique words and converts sentences into numerical vectors based on their occurrences.

Applications

- Document classification, spam filtering, and sentiment analysis.
- Search engines for keyword matching.
- Basic NLP models where context is not required.
- Traditional ML models that rely on BoW features.

Example

Sentence: "I love AI" → Vocabulary: [I, love, AI] → Vector: `[1, 1, 1]`

1.2.2 TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF measures how important a word is in a document relative to a collection of documents. It increases the weight of words that appear frequently in a document but reduces the weight of very common words across all documents.

Applications

- Information retrieval systems and search engines.
- Document ranking and keyword extraction.
- Text classification and clustering.

- Baseline feature method in NLP pipelines.

Example

In "AI is powerful," **"AI"** receives a higher TF-IDF score than the common word **"is"**, making it more significant in the representation.

1.3 Prediction-Based Embeddings

Prediction-based embeddings are advanced word vectorization techniques that use neural networks to learn word representations based on their context in a sentence. Instead of counting word frequency, these methods predict a word from its surrounding words (or vice versa), capturing semantic meaning and relationships.

1.3.1 Word2Vec

Word2Vec uses neural networks to learn word meanings based on context. It represents words as dense vectors where similar words have similar representations.

- Identifies semantic similarities and word analogies.
- Applied in recommendation systems and search engines.
- Enhances chatbots and machine translation.

"king − man + woman ≈ queen" — classic analogy demonstrating semantic relationships.

1.3.2 GloVe (Global Vectors)

GloVe learns word representations using global word co-occurrence statistics from a corpus, capturing both semantic meaning and relationships by analyzing how often words appear together.

- Sentiment analysis and text classification.
- Document analysis and information retrieval.
- Question answering and natural language understanding.

Words like "ice" and "cold" share similar vectors because they frequently co-occur in similar contexts.

1.3.3 FastText

FastText, developed by Facebook, represents words using subword (character n-gram) information, allowing it to handle rare and unseen words effectively. It captures morphological structure and context.

- Text classification in large datasets and multilingual environments.
- Effective with rare and out-of-vocabulary (OOV) words.
- Language detection, spam filtering, and search systems.

Even if "unhappiness" was never seen in training, FastText understands it from subword parts: "un", "happy", "ness".

1.4 Key Comparison of Word Vectorization Methods

Feature	BoW	TF-IDF	Word2Vec (CBOW/Skip-gram)	GloVe	FastText
Type	Frequency	Weighted frequency	Neural embedding	Statistical embedding	Subword embedding
Context	✗No	✗No	✓Yes	✓Yes	✓Yes
Semantic Meaning	✗No	✗Limited	✓Yes	✓Yes	✓Yes
Vector Form	Sparse	Sparse	Dense	Dense	Dense
OOV Words	✗No	✗No	✗No	✗No	✓Yes
Performance	Low	Medium	High	High	Very High

PART 2 — IMPLEMENTATION & RESULTS

2.1 Dataset Description

Dataset Name	IMDB Top 1000 Movies and TV Shows
Source	Kaggle — harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows
Total Samples	1,000 records
Features Used	Overview, Genre, Director, Stars (1–4), Certificate
Target Label	Binary — IMDB Rating above/below median (0 = Negative, 1 = Positive)
Class Distribution	Negative (0): 537 samples | Positive (1): 463 samples

2.2 Implementation Code

The following Python code implements the complete pipeline: dataset loading, text preprocessing, feature vectorization (BoW and TF-IDF), hyperparameter tuning via Grid Search, and evaluation.

```
# ==================================================
# IMPORT LIBRARIES
# ==================================================
import kagglehub
from kagglehub import KaggleDatasetAdapter
import pandas as pd, nltk, re, time
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, classification_report)

# ==================================================
# LOAD DATASET
# ==================================================
df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows",
    "imdb_top_1000.csv"
)

# ==================================================
# TEXT PREPROCESSING
# ==================================================
stop_words = set(stopwords.words("english")) - {"not", "no", "never"}
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z\s]", " ", text)
```

```
    text = re.sub(r"\s+", " ", text).strip()
    tokens = [lemmatizer.lemmatize(w) for w in text.split()
              if w not in stop_words and len(w) > 2]
    return " ".join(tokens)

# ==================================================
# FEATURE ENGINEERING & LABEL CREATION
# ==================================================
text_cols =
["Overview","Genre","Director","Star1","Star2","Star3","Star4","Certificate"]
text_cols = [c for c in text_cols if c in df.columns]
df[text_cols] = df[text_cols].fillna("")
df["combined_text"] = df[text_cols].astype(str).agg(" ".join, axis=1)
df["clean_text"]    = df["combined_text"].apply(preprocess)

threshold    = df["IMDB_Rating"].median()
df["label"]  = (df["IMDB_Rating"] > threshold).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    df["clean_text"], df["label"], test_size=0.2, random_state=42,
stratify=df["label"]
)

# ==================================================
# TF-IDF PIPELINE WITH GRID SEARCH
# ==================================================
tfidf_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=30000, ngram_range=(1,2),
                              min_df=2, max_df=0.9, sublinear_tf=True)),
    ("model", LinearSVC())
])
tfidf_grid = GridSearchCV(tfidf_pipeline, {"model__C":[0.1,0.5,1,2,5]},
                          cv=5, scoring="f1", n_jobs=-1)
start = time.time(); tfidf_grid.fit(X_train, y_train)
tfidf_time = time.time() - start

# ==================================================
# BAG OF WORDS PIPELINE WITH GRID SEARCH
# ==================================================
bow_pipeline = Pipeline([
    ("bow", CountVectorizer(max_features=30000, ngram_range=(1,2),
                            min_df=2, max_df=0.9, binary=True)),
    ("model", LinearSVC())
])
bow_grid = GridSearchCV(bow_pipeline, {"model__C":[0.1,0.5,1,2,5]},
                        cv=5, scoring="f1", n_jobs=-1)
start = time.time(); bow_grid.fit(X_train, y_train)
bow_time = time.time() - start
```

2.3 TF-IDF Results

Best Parameter (C)	To be filled from output
Accuracy	0.56
Precision	0.538
Recall	0.376

F1-Score	0.443
Execution Time	2.84 seconds

Classification Report — TF-IDF

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.57	0.72	0.64	107
Positive (1)	0.54	0.38	0.44	93

2.4 Bag of Words Results

Best Parameter (C)	To be filled from output
Accuracy	0.545
Precision	0.515
Recall	0.376
F1-Score	0.435
Execution Time	1.76 seconds

Classification Report — Bag of Words

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.56	0.69	0.62	107
Positive (1)	0.51	0.38	0.43	93

2.5 Output Graphs

The following graphs illustrate model performance metrics and comparative results across both vectorization methods.

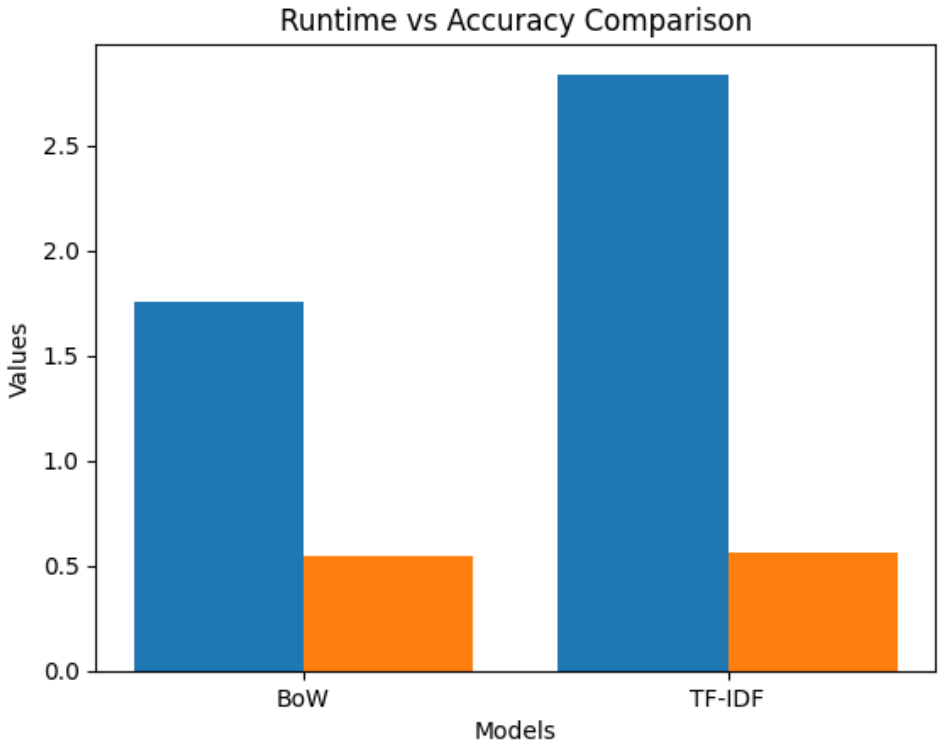

Figure 1 — Model Performance Comparison (Graph 1)

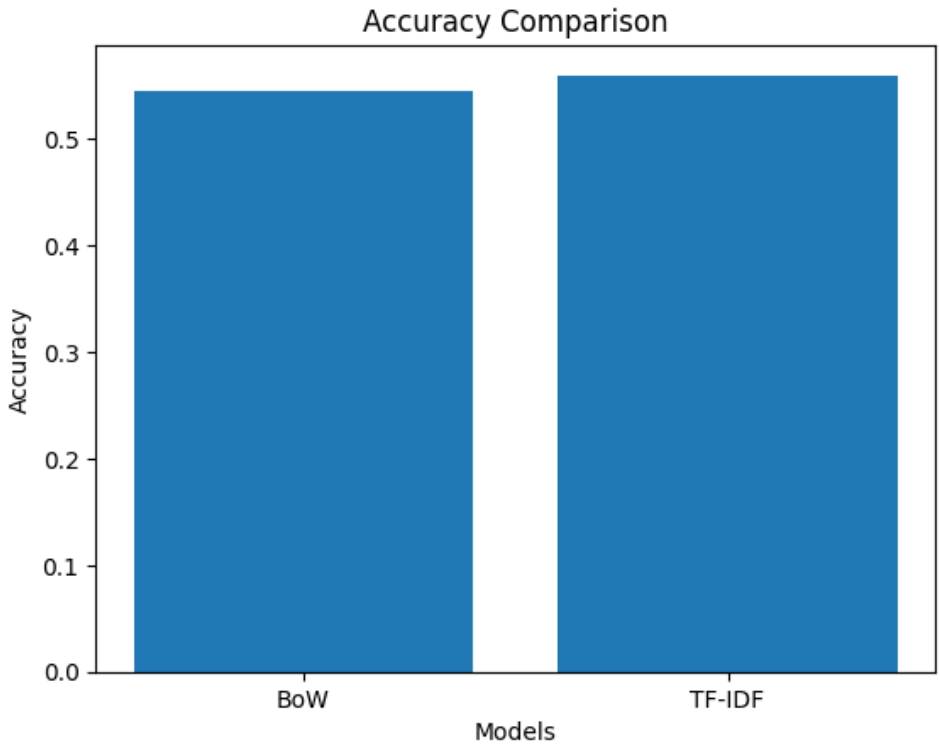

Figure 2 — Model Performance Comparison (Graph 2)

PART 3 — INTERPRETATION & ANALYSIS

3.1 Performance vs. Time Complexity Trade-off

Bag of Words (BoW) offers low time complexity since it simply counts word frequencies, making it fast and computationally efficient. However, this simplicity results in lower performance as it treats all words equally and ignores their relative importance.

In contrast, TF-IDF introduces additional computation by weighting words based on their frequency and rarity across documents, which increases processing time. This leads to better performance because it highlights more meaningful words and reduces the influence of common terms.

Dimension	BoW	TF-IDF
Time Complexity	Lower — simple frequency count	Higher — additional IDF computation
Accuracy	0.545 (lower)	0.560 (slightly better)
F1-Score	0.435	0.443
Execution Time	1.76 seconds	2.84 seconds
Context Capture	None — bag of words only	None — frequency-weighted only
Suitability	Quick baseline; large corpora	Better relevance; NLP pipelines

3.2 Key Findings and Insights

- **Finding 1:** Both BoW and TF-IDF provide comparable performance, with TF-IDF achieving only a marginal improvement in accuracy (0.56 vs. 0.545) and F1-Score (0.443 vs. 0.435).
- **Finding 2:** Both models performed better on the negative class (precision 0.57/0.56) than the positive class (0.54/0.51), suggesting a class bias or difficulty in capturing positive sentiment patterns.
- **Finding 3:** BoW demonstrated lower computational time (1.76 s) compared to TF-IDF (2.84 s). The performance gain of TF-IDF is relatively small compared to its increase in time complexity.
- **Finding 4:** Neither method captures semantic meaning or contextual information, fundamentally limiting overall effectiveness on sentiment tasks. Prediction-based embeddings (Word2Vec, GloVe, FastText) would likely produce significantly better results.
- **Finding 5:** The consistently low recall for the positive class (0.38 for both methods) indicates that many positively rated movies are being misclassified — a reflection of the models' inability to understand nuanced language.

3.3 Conclusion

> ***Traditional vectorization methods provide efficient but shallow text representations, leading to limited performance improvements.***

The experiment clearly demonstrates the trade-off between efficiency and performance. BoW is the faster choice for low-resource environments with no significant accuracy penalty for many tasks, while TF-IDF offers marginally better feature representation at a slightly higher computational cost.

For this IMDB classification task, neither method is sufficient on its own. Future work should incorporate prediction-based embeddings (Word2Vec, GloVe, FastText, or transformer-based models like BERT) to capture deeper semantic relationships and achieve substantially improved classification performance.